Lecture 4: Assembly

Professor G. Sandoval

04

NYU

Some slides adapted by G. Sandoval for CS3224, from slide by Brendan Dolan-Gavitt and Bryant and O'Hallaron, Computer Systems: A programmer's Perspective, Third Edition

Today's questions

- What state does the CPU keep?
- How do instructions read and change memory?
- How do branches and function calls change control flow?

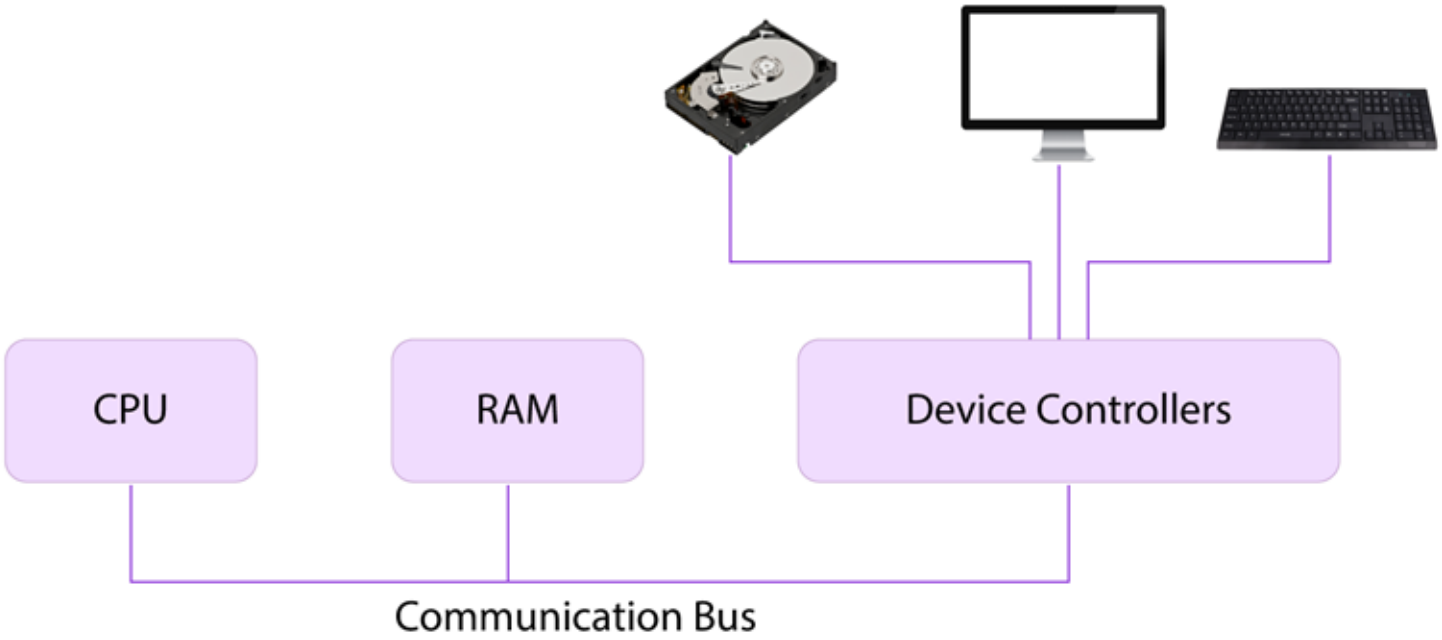
NYU

Our machine model

- We use 32-bit x86 and AT&T assembly syntax.
- A register is a small storage location inside the CPU.
- Memory is byte-addressed. An address identifies a location.
- EIP tracks control flow. EFLAGS holds status and control bits.

PC Architecture

(Vastly simplified)

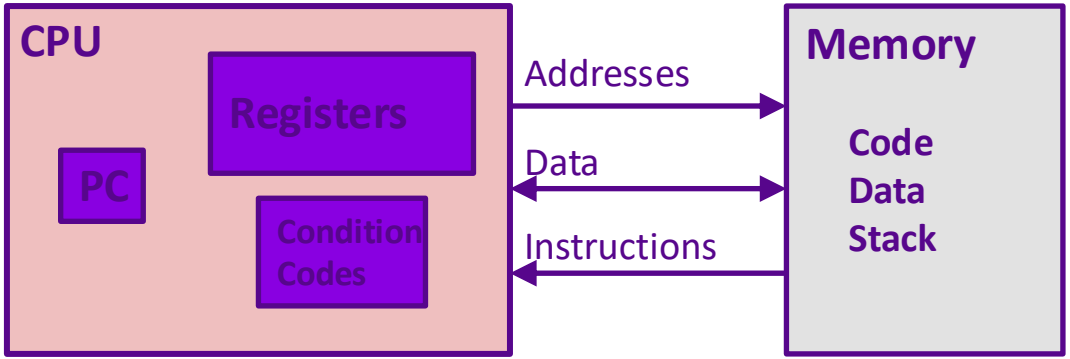


NYU

Fetch, decode, execute

- Fetch the instruction at the current instruction pointer.
- Decode the operation and its operands.
- Execute it and update the relevant machine state.
- Continue at the next instruction, unless control flow changes.

Assembly/Machine Code View



PC: next instruction address

Registers: working values

Flags: operation status

Memory holds code and data

The stack supports calls

Registers, cache, and memory

- Registers hold operands the CPU can use directly.
- Caches keep copies of recently used memory data.
- RAM holds the working memory of running programs.
- SSDs and disks provide persistent storage with much higher access latency.

Which access is fastest? For an operand already present in each location, which usually has the lowest access latency?

A CPU register

0%

An SSD

0%

Main memory (RAM)

0%

A CPU cache

0%

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Answer: a CPU register

- A register already contains the operand inside the CPU.
- A cache hit avoids a slower trip to main memory.
- An SSD access involves persistent storage and an I/O path.
- Why can a program still spend time waiting for data?

CPU

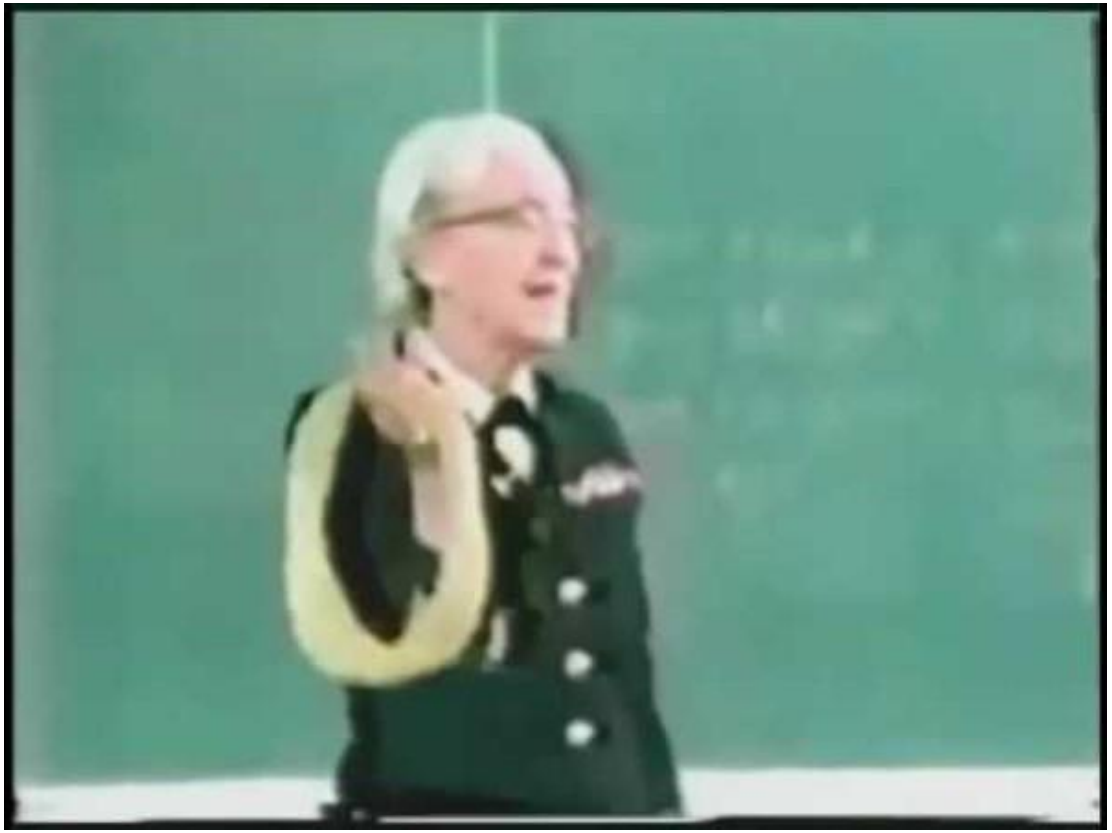
- **Contains a (tiny) bit of local storage (registers)**
  - ~1-3 nanoseconds to access
- **Contains a little more on-chip storage (cache)**
  - ~2-8 ns to access L1, 5-12 ns to access L2
- **For comparison:**
  - RAM is 10-60 ns,
  - SSD is 100 µs, (microseconds)
  - disk is 3-20 **ms**

Nanosecond = billionth of a second = 10^-9

microsec = 1 millionth of a second = 10^-6

millisecond = 1 thousand of a second = 10^-3

ASIDE: Grace Hopper: Nanosecond vs Millisecond.



NYU

Boot firmware

- The CPU begins executing firmware after reset.
- Firmware initializes enough hardware to locate boot code.
- It loads the next stage and transfers control.
- Our legacy PC model uses BIOS. Other systems commonly use UEFI.

Devices and address spaces

- Memory-mapped I/O assigns addresses to device registers.
- Port I/O uses instructions such as in and out.
- Interrupts let devices request CPU attention.
- The OS controls which operations a user program may perform.

Example PC physical address map

Start	End	Description
0x00000000	0x0009FFFF	Low Memory
0x000A0000	0x000BFFFF	VGA Memory
0x000C0000	0x000EFFFF	16-bit Devices
0x000F0000	0x000FFFFF	BIOS ROM
0x00100000	?	Extended Memory
…		
0xC0000000	0xFFDFFFFF	PCI Devices
0xFFE00000	0xFFFFFFFF	High BIOS

On Linux, you can look at /proc/iomem for a detailed list

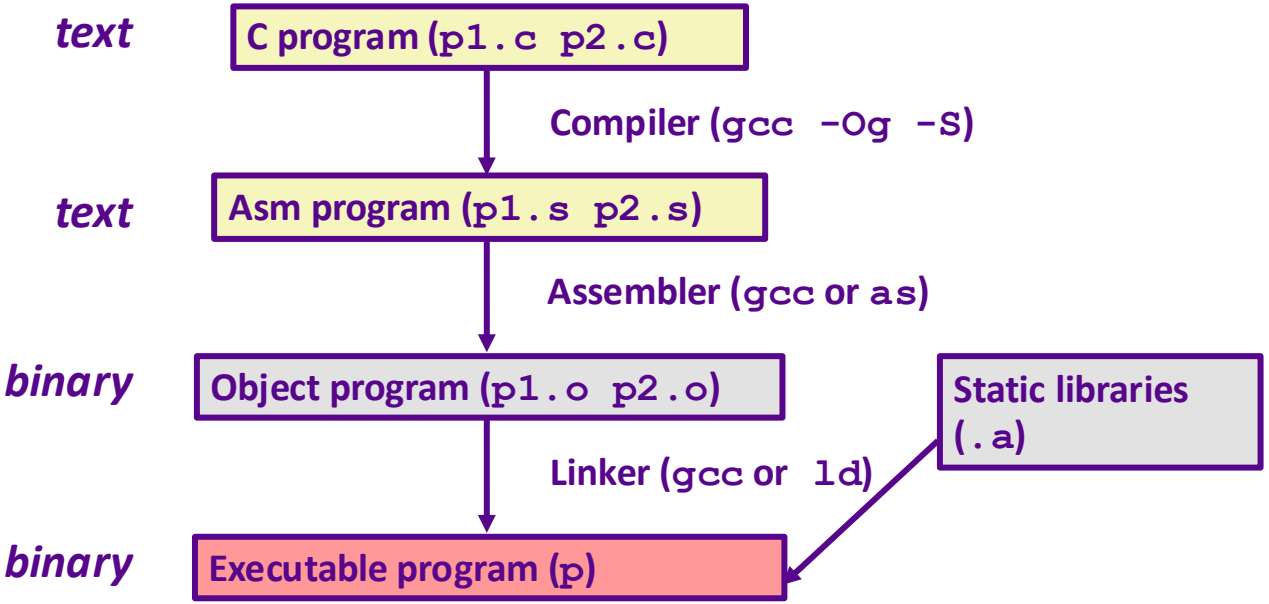
Modes and segmentation

- Protected mode supports isolation and address translation.
- A 32-bit linear address space contains 4 GiB of byte addresses.
- A segment selector identifies a descriptor with a base and access rules.
- For our traces, assume flat segments with base zero.

Turning C into Object Code

Code in files `p1.c p2.c`

Compile with command: `gcc -Og p1.c p2.c -o p`



x86 (32-bit)

- General purpose registers:

  EAX, EBX, ECX, EDX, ESI, EDI

  (though some have historical uses: ECX is used for counters, ESI & EDI are source and destination for string operations)
- Special-purpose registers:

  EBP (base pointer), **ESP** (stack pointer)
- Flags register: **EFLAGS**
- Segment registers: **CS, SS, DS, ES, FS, GS**
- Instruction pointer/program counter: **EIP**
- Control registers: **CR0-CR4**

Addressable Memory

- **Protected Mode**
  - 4 GB Address space
  - 32 bit addresses

- **Real address and Virtual 8086 Modes**
  - 1 MB address space
  - 20 bit addresses

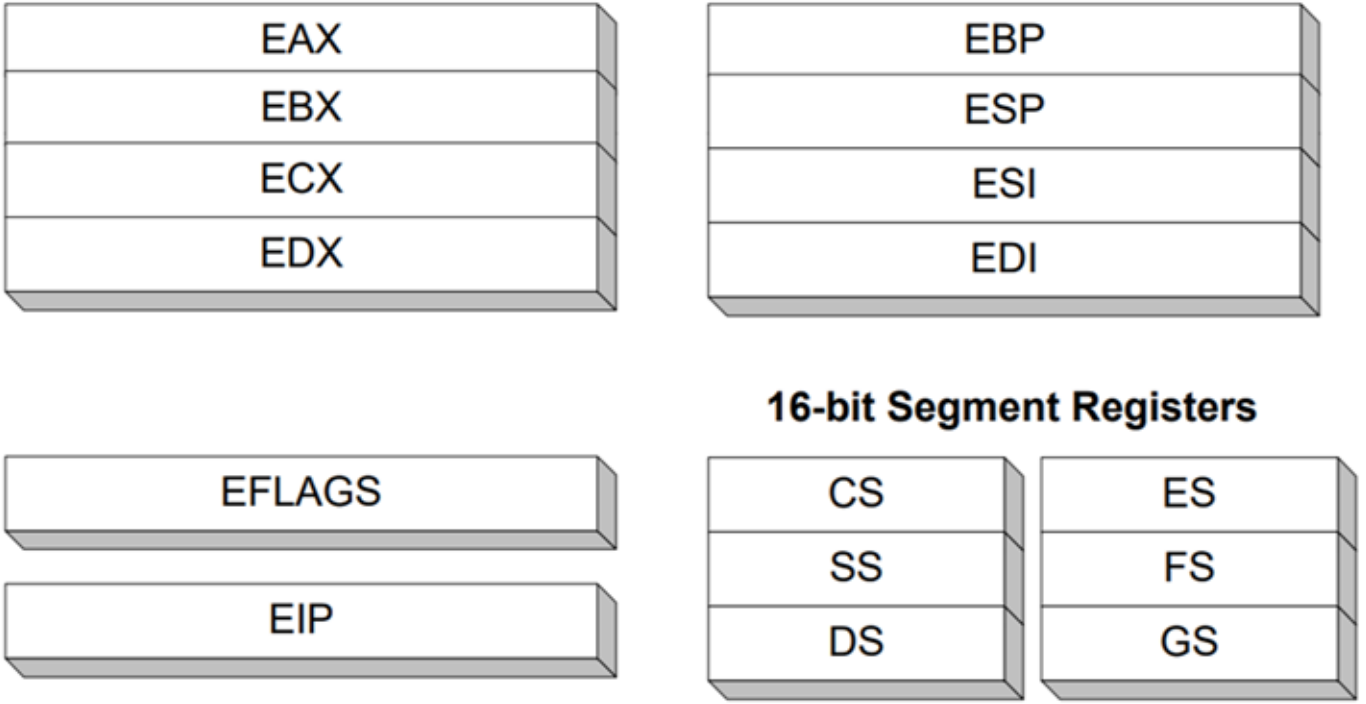
NYU

x86 Assembly Syntax

- **Two "flavors":**
  - Intel syntax (what's in the processor manual)
  - AT&T syntax (what GNU as uses)
- **The difference between them:**
  - Intel: mov eax, DWORD PTR [ebx] -> **DEST, SRC**
  - AT&T: movl (%ebx), %eax -> **SRC, DEST**

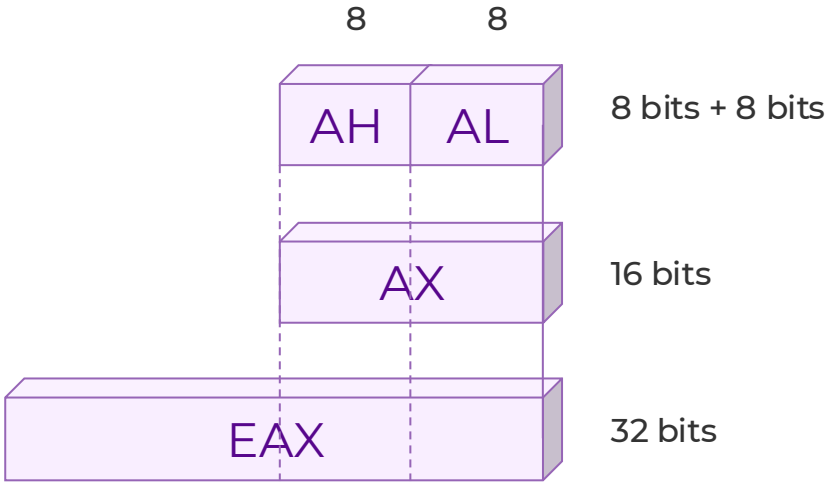
We will work with **AT&T syntax**

32 bit General Purpose Registers



Accessing parts of registers

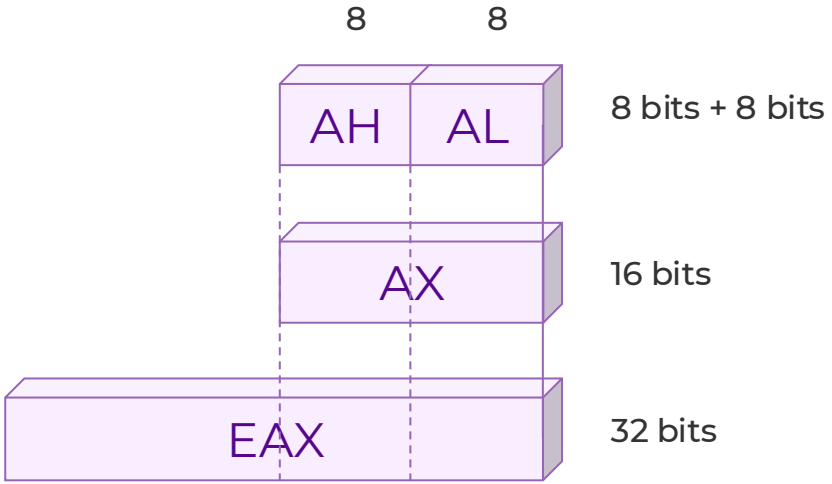
- AL is the low 8 bits of EAX.
- AH is the next 8 bits.
- AX is the low 16 bits.



NYU

Accessing parts of registers (2)

32-bit	16-bit	8-bit (high)	8 bit (low)
EAX	AX	AH	AL
EBX	BX	BH	BL
ECX	CX	CH	CL
EDX	DX	DH	DL



NYU

AT&T syntax

- Two-operand instructions usually **write the destination last**.
- **%** names a **register.**
- **$** marks an immediate **constant.**
- **Parentheses** around a register request a **memory operand.**
- The **suffix b** selects **8 bits**.
- The **suffix l** selects **32 bits.**

Data Movement

- **mov** <reg>,<reg>
- **mov** <reg>,<mem>
- **mov** <mem>,<reg>
- **mov** <reg>,<const>
- **mov** <mem>,<const>

- What's missing?

Data Movement

Examples (AT&T syntax)

- Source, dest
- **movl** %eax, %edx => edx = eax
- **movl** $0x123, %ebx => ebx = 0x123
- **movl** 0x123, %ebx => ebx = *(int *)0x123
- **movl** (%ebx), %edx => edx = *(int *)ebx
- **movl** 4(%ebx), %edx =>
    - edx = *(int *)(ebx + 4)

Data movement

```
movl $7, %eax
movl %eax, %ebx
movl (%ecx), %edx
movl %edx, (%ecx)
```

A constant, a register copy, a load, and a store

Pair trace: one byte changes

Initial EAX = 0x12345678

```
movb $0xAA, %al
```

What are **EAX** and **AX** afterward?

Which bits stay unchanged?

Predict first, then compare

NYU

Addresses and values

%eax	value in EAX
$0x108	constant 0x108
0x104	memory at address 0x104
(%eax)	memory at address EAX
4(%eax)	memory at address EAX + 4

The displacement counts bytes

Pair trace: addresses and values

Address	Value	Register	Value
0x100	0xFF	%eax	0x100
0x104	0xAB	%ecx	0x1
0x108	0x13	%edx	0x3
0x10C	0x11		

Operand	Value	Comment
%eax		
0x104		
$0x108		
(%eax)		
4(%eax)		

movl SOURCE, %ebx

lea computes an address

EAX = 0x1000, memory[0x1004] = 9

movl 4(%eax), %ebx

leal 4(%eax), %ecx

movl loads 9 into EBX.

leal computes 0x1004 and puts it in ECX.

EAX = 0x2000, memory[0x2008] = 12 movl 8(%eax), %ebx leal 8(%eax), %ecx Which pair is (EBX, ECX)?

(0x2008, 0x2008) 0%

(12, 12) 0%

(12, 0x2008) 0%

(0x2008, 12) 0%

None of the above 0%

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Pair repair: a memory copy

movl (%eax), (%ebx)

Copy one 32-bit value from the address in EAX to the address in EBX.

- Why is this instruction invalid?
- Repair it using EDX as a scratch register.

Arithmetic

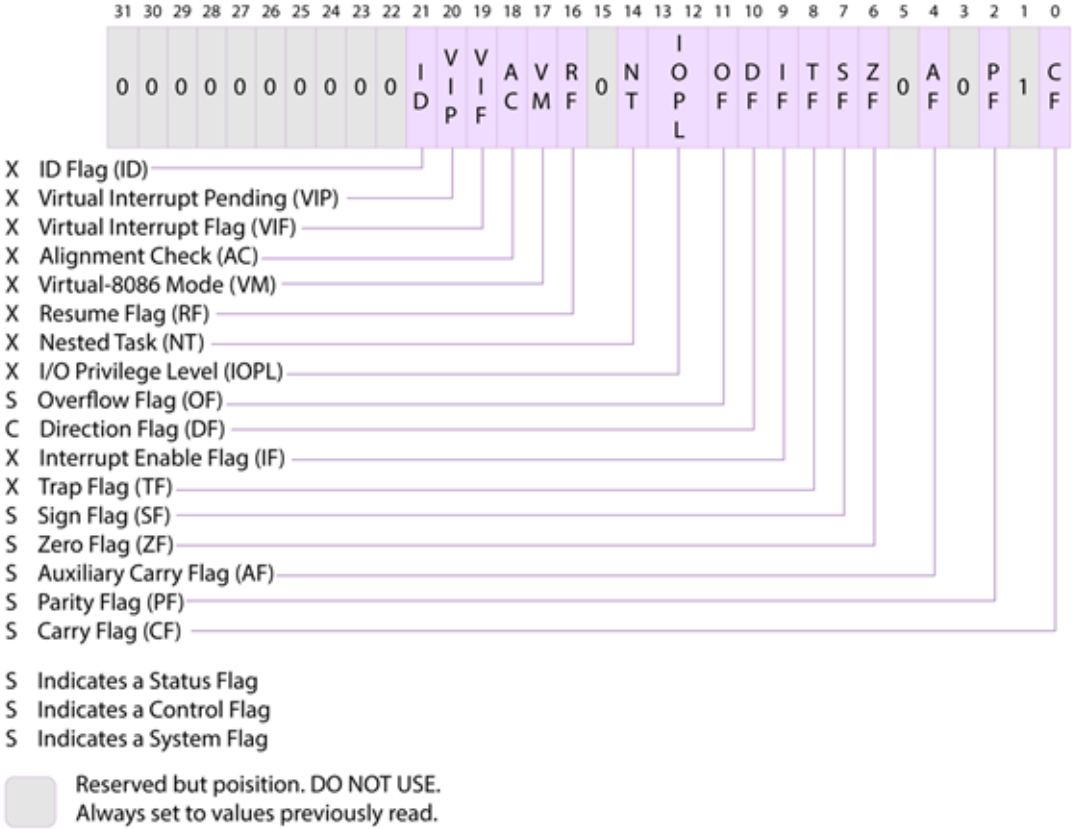
- **addl** adds the source to the destination.
- **subl** subtracts the source from the destination.
- **andl**, **orl**, and **xorl** operate on bits.
- **shll** and **shrl** shift bits left or right.

cmp and test

- **cmpl SRC**, **DST** sets flags for DST minus SRC.
- **testl SRC, DST** sets flags for their bitwise AND.
- Both leave the operands unchanged.
- A conditional branch reads those flags.

EFLAGS

ZF: zero
SF: sign
CF: carry or borrow
OF: signed overflow



EFLAGS Register

Conditional branches

- **je / jz** tests equality or a zero result.
- **jne / jnz** tests inequality or a nonzero result.
- **jl:** signed less. **jg:** signed greater.
- **jb**: unsigned below**. ja:** unsigned above.

NYU

Predict: which branches are taken?

```
movl $-1, %eax
cmpl $1, %eax
```

Consider each branch separately after this comparison. target names a code address.

jl target

jb target

Answer: jl is taken, jb is not

- Signed: -1 is less than 1, so jl jumps.
- Unsigned: 0xFFFFFFFF is greater than 1, so jb does not jump.
- The bit pattern is the same.
- The branch instruction chooses how to interpret the comparison.

Labels and jumps

```
.Lagain:
    addl $1, %eax
    cmpl $3, %eax
    jl .Lagain
```

A code label names an instruction address.

jmp always transfers control. A conditional jump tests a condition.

Pair trace: what does this loop compute?

```
movl $0, %eax
movl $3, %ecx
.Lloop:
    addl %ecx, %eax
    subl $1, %ecx
    jnz .Lloop
```

Write (EAX, ECX) after each iteration.

Which instruction sets the flags used by jnz?

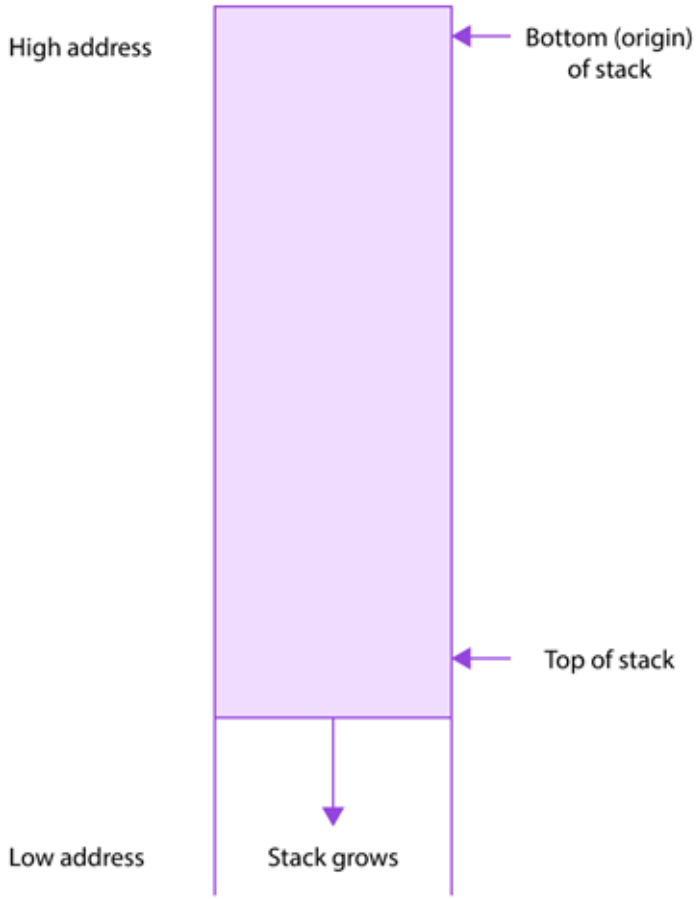
NYU

pushl and popl

- **ESP** holds the address of the top stack item.
- **pushl** subtracts 4 from ESP, then writes a 32-bit value.
- **popl** reads a 32-bit value, then adds 4 to ESP.
- The active stack grows toward lower addresses.

The stack in x86

- ESP points to the active top item.
- pushl moves ESP to a lower address.
- popl moves ESP to a higher address.



NYU

Pair trace: the stack pointer

Initial ESP = 0x1000

pushl $7

pushl $9

popl %eax

What are EAX and ESP afterward?

What value is now at the top of the stack?

NYU

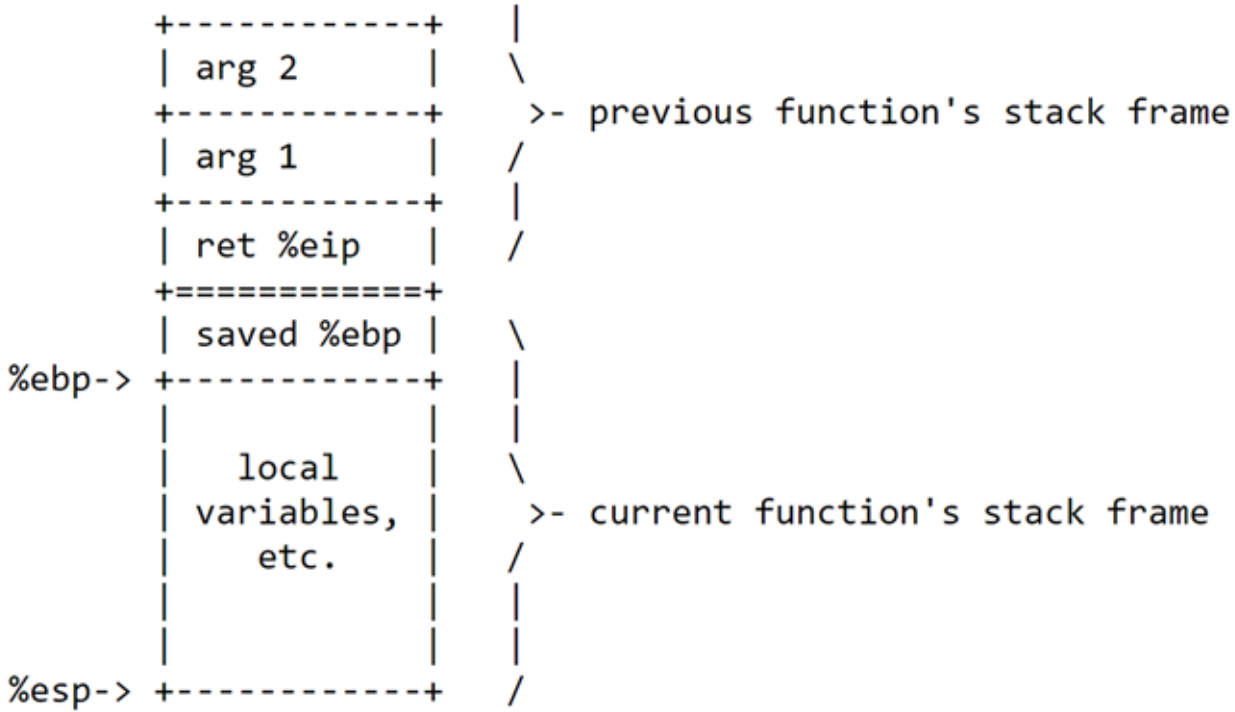
call and ret

- A **call** pushes the address of the following instruction.
- It then jumps to the function's entry point.
- **ret** pops that saved address into the instruction pointer.
- An ordinary call **does not change** the privilege level.

Stack frames

- A function may need arguments, saved registers, and local storage.
- Its stack frame organizes those values.
- In our 32-bit calling convention, callers put arguments on the stack.
- EBP can provide a stable reference while ESP changes.

A conventional 32-bit stack frame



A function prologue

```
pushl %ebp
movl %esp, %ebp
subl $8, %esp
```

- Save the caller's frame pointer.
- Establish this frame's base and reserve 8 bytes for locals.

A function epilogue

```
movl %ebp, %esp
popl %ebp
Ret
```

- Release the local stack area.
- Restore the caller's EBP and return to the saved code address.

trace: a function argument

0(%ebp) 0%

4(%ebp) 0%

8(%ebp) 0%

-4(%ebp) 0%

None of the above 0%

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

String instructions

- movsb copies a byte from (ESI) to (EDI).
- The direction flag controls whether the pointers increase or decrease.
- cld selects increasing addresses.
- With 32-bit addressing, rep movsb uses ECX as its repeat count.

Assembly at the OS boundary

- The OS uses machine state to suspend and resume execution.
- System-call entry transfers control through a permitted kernel entry point.
- xv6 x86 uses int for that entry and iret for the return path.
- Saving and restoring state makes controlled execution possible.

Exit trace: a call and return

```
Initial ESP = 0x1000
pushl $7
call f
.Lafter:
    addl $4, %esp
```

f loads its first stack argument into EAX, adds 1, then returns.

Exit Poll

```
Caller:
        ESP=0x1000,
        push 7,
        call f, then
        ESP += 4

f:
    movl 4(%esp), %eax
    addl $1, %eax
    ret
```

After the caller executes

ESP += 4 what are (EAX, ESP)?

7, 0x1000 — 0%

8, 0x1000 — 0%

8, 0x0FFC — 0%

8, 0x0FF* — 0%

None of the above — 0%

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**